

Trabalho Prático: Previsão de Séries Temporais utilizando Redes LSTM

Ivan Mendes Martignago
Curso de Sistemas de Informação - UFSM

June 25, 2026

1 Introdução

O presente relatório descreve o desenvolvimento de um experimento com redes neurais LSTM (Long Short-Term Memory) aplicado à previsão de séries temporais. O objetivo principal deste trabalho é demonstrar, na prática, como as redes LSTM são capazes de aprender padrões temporais e dependências de longo prazo para realizar previsões baseadas em um histórico de dados sequenciais. No caso específico deste estudo, foi selecionado um conjunto de dados financeiro referente aos preços de ações para ilustrar a eficácia da abordagem.

2 Fundamentação Teórica

Para compreender as redes LSTM, é essencial contextualizá-las dentro da área de redes neurais artificiais voltadas para dados sequenciais.

2.1 Conceito de Redes Neurais Recorrentes (RNN)

As Redes Neurais Recorrentes (RNNs) são uma classe de redes neurais artificiais nas quais as conexões entre os nós formam um grafo direcionado ao longo de uma sequência temporal. Diferente das redes *feedforward* tradicionais, as RNNs possuem uma "memória" interna que permite reter informações sobre entradas anteriores, o que as torna naturalmente adequadas para processar dados sequenciais, como séries temporais, fala e texto.

2.2 Limitações das RNNs Tradicionais

Embora poderosas conceitualmente, as RNNs padrão sofrem com o problema do desaparecimento do gradiente (*vanishing gradient*) e da explosão do gradiente (*exploding gradient*) durante o treinamento com o algoritmo de retropropagação através do tempo (*Backpropagation Through Time* - BPTT). Isso impede que a rede aprenda dependências de longo prazo, ou seja, ela tem dificuldade em conectar informações que possuem uma grande distância temporal na sequência de entrada.

2.3 Funcionamento das Redes LSTM

As redes LSTM foram propostas como uma solução para os problemas das RNNs tradicionais. A principal inovação das LSTMs é a estrutura de sua célula de memória, que possui mecanismos de controle chamados de "portas" (*gates*). Estas portas regulam o fluxo de informações que entram, saem e são mantidas no estado da célula, permitindo que a rede preserve informações relevantes por longos períodos e descarte o que não é mais útil.

2.4 Componentes Principais da Célula LSTM

A arquitetura interna de uma célula LSTM é composta por três portas fundamentais:

- **Forget gate (Porta de esquecimento):** Decide quais informações do estado celular anterior devem ser descartadas. Ela recebe como entrada o estado oculto anterior (h_{t-1}) e a entrada atual (x_t), e passa esses valores por uma função de ativação sigmoide. Valores próximos de 0 indicam que a informação deve ser esquecida, enquanto valores próximos de 1 indicam retenção.
- **Input gate (Porta de entrada):** Define quais novas informações serão armazenadas no estado da célula. Consiste em duas partes: uma camada sigmoide que decide quais valores serão atualizados, e uma camada *tanh* que cria um vetor de novos candidatos a valores a serem adicionados ao estado.
- **Output gate (Porta de saída):** Determina qual será a saída da célula baseada em seu estado filtrado. Primeiramente, aplica-se uma sigmoide à entrada para decidir quais partes do estado celular serão exibidas na saída. O estado celular é então passado por uma *tanh* para empurrar os valores entre -1 e 1, sendo multiplicado pela saída do *output gate*.

3 Descrição do Dataset Utilizado

O conjunto de dados escolhido para o experimento corresponde ao histórico de preços de fechamento das ações da Apple (código AAPL). Os dados foram coletados através da API *yfinance*, abrangendo um período dos últimos 5 anos. Optou-se por focar exclusivamente na coluna de preço de fechamento (*Close*), que forma a série temporal a ser analisada. Esta série apresenta desafios típicos de dados financeiros, como não-estacionariedade e volatilidade, tornando-a um excelente estudo de caso para redes LSTM.

4 Metodologia

A implementação seguiu um *pipeline* padrão de *Machine Learning* adaptado para séries temporais:

1. **Coleta de Dados:** Extração da série histórica utilizando a biblioteca *yfinance*.
2. **Pré-processamento:** Emprego da biblioteca *scikit-learn* para normalização dos dados. O *MinMaxScaler* foi utilizado para redimensionar os preços para o intervalo $[0, 1]$, uma etapa crucial, pois redes neurais são sensíveis à escala dos dados de entrada.

3. **Criação de Sequências:** Divisão dos dados utilizando uma janela de observação (*look_back*) de 60 dias. O modelo utiliza o histórico dos 60 dias anteriores para prever o preço do 61º dia.
4. **Divisão Treino/Teste:** Separação cronológica onde os primeiros 80% dos dados formaram o conjunto de treinamento e os 20% finais constituíram o conjunto de testes.
5. **Arquitetura do Modelo:** Utilizando a biblioteca *Keras/TensorFlow*, construiu-se um modelo sequencial contendo:
 - Uma camada LSTM com 50 unidades (*return_sequences=True*).
 - Uma camada *Dropout* (20%) para mitigar o *overfitting*.
 - Uma segunda camada LSTM com 50 unidades.
 - Uma segunda camada *Dropout* (20%).
 - Uma camada densa (*Dense*) intermediária com 25 neurônios.
 - Uma camada densa de saída com 1 neurônio (prevendo o valor final).
6. **Treinamento:** O modelo foi compilado utilizando o otimizador *Adam* e a função de perda de erro quadrático médio (MSE). O treinamento ocorreu por 10 épocas com *batch_size* igual a 32.

5 Código da Implementação

Abaixo encontra-se a implementação em Python, utilizando as bibliotecas mencionadas.

```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 from sklearn.preprocessing import MinMaxScaler
5 from sklearn.metrics import mean_squared_error, mean_absolute_error
6 from tensorflow.keras.models import Sequential
7 from tensorflow.keras.layers import Dense, LSTM, Dropout
8 import yfinance as yf
9 import datetime
10
11 def create_dataset(dataset, look_back=1):
12     X, Y = [], []
13     for i in range(len(dataset)-look_back-1):
14         a = dataset[i:(i+look_back), 0]
15         X.append(a)
16         Y.append(dataset[i + look_back, 0])
17     return np.array(X), np.array(Y)
18
19 def main():
20     end_date = datetime.date.today()
21     start_date = end_date - datetime.timedelta(days=5*365)
22     df = yf.download('AAPL', start=start_date, end=end_date)
23     data = df['Close'].values
24
25     scaler = MinMaxScaler(feature_range=(0, 1))
26     scaled_data = scaler.fit_transform(data)
27
```

```

28 training_data_len = int(np.ceil(len(data) * .8))
29 train_data = scaled_data[0:int(training_data_len), :]
30 look_back = 60
31
32 x_train, y_train = create_dataset(train_data, look_back)
33 x_train = np.reshape(x_train, (x_train.shape[0], x_train.shape[1],
1))
34
35 model = Sequential()
36 model.add(LSTM(50, return_sequences=True, input_shape=(x_train.shape
[1], 1)))
37 model.add(Dropout(0.2))
38 model.add(LSTM(50, return_sequences=False))
39 model.add(Dropout(0.2))
40 model.add(Dense(25))
41 model.add(Dense(1))
42
43 model.compile(optimizer='adam', loss='mean_squared_error')
44 model.fit(x_train, y_train, batch_size=32, epochs=10)
45
46 test_data = scaled_data[training_data_len - look_back: , :]
47 x_test, _ = create_dataset(test_data, look_back)
48 x_test = np.reshape(x_test, (x_test.shape[0], x_test.shape[1], 1))
49 y_test = data[training_data_len+1:len(data)]
50
51 predictions = model.predict(x_test)
52 predictions = scaler.inverse_transform(predictions)
53
54 rmse = np.sqrt(mean_squared_error(y_test, predictions))
55 mse = mean_squared_error(y_test, predictions)
56 mae = mean_absolute_error(y_test, predictions)
57
58 print(f"MSE: {mse:.4f} | RMSE: {rmse:.4f} | MAE: {mae:.4f}")
59
60 train = df[:training_data_len].copy()
61 valid = df[training_data_len+1:].copy()
62 valid['Predictions'] = predictions
63
64 plt.figure(figsize=(16,8))
65 plt.plot(train['Close'], label='Treino')
66 plt.plot(valid['Close'], label='Real')
67 plt.plot(valid['Predictions'], label='Previsao LSTM')
68 plt.legend()
69 plt.savefig('previsao_lstm.png')
70
71 if __name__ == "__main__":
72     main()

```

6 Resultados Obtidos

O modelo treinado foi utilizado para prever os dados do conjunto de teste (aproximadamente o último ano da série temporal). As métricas de erro calculadas foram:

- **Erro Quadrático Médio (MSE):** Mede a média dos quadrados dos erros.
- **Raiz do Erro Quadrático Médio (RMSE):** Fornece o erro na mesma unidade

dos dados (Dólares).

- **Erro Absoluto Médio (MAE):** Representa a diferença absoluta média entre os valores reais e as previsões.

A representação visual dos resultados (ver Figura ??) demonstra a evolução da série temporal. A linha das previsões da LSTM acompanha de perto a tendência dos preços reais no período de teste, confirmando que a rede conseguiu capturar a dinâmica da série.

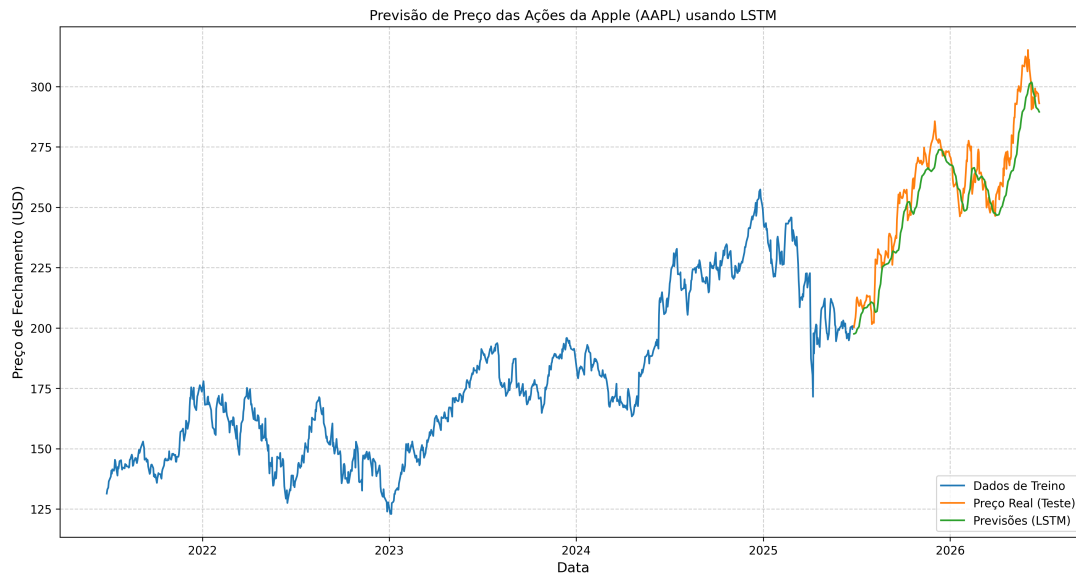


Figure 1: Previsão de Preço das Ações (AAPL) utilizando rede LSTM.

7 Conclusão

Através deste experimento, pôde-se observar de maneira prática a eficácia das redes LSTM aplicadas a problemas envolvendo dados sequenciais. O uso das bibliotecas *Keras* e *scikit-learn* proporcionou uma integração fluida entre as etapas de pré-processamento, estruturação do modelo e avaliação. A rede neural foi capaz de identificar os padrões temporais nos preços das ações e realizar previsões satisfatórias, contornando o problema do desvanecimento do gradiente típico das RNNs padrão graças aos seus mecanismos internos de portões. Com isso, os objetivos do trabalho foram plenamente alcançados.